COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 Data Structures and Algorithms	**SECTION:**	CPE 21S2
DATE PERFORMED:	September 02, 2026	**DATE SUBMITTED:**	September 02, 2026
NAME:	Mark Joneil Bautista	**INSTRUCTOR:**	Engr. Jimlord Quejado

Seatwork 6.1 **Searching Algorithm**

1. PROCEDURE OUTPUT

Table 6-1 Data Generated and Observations

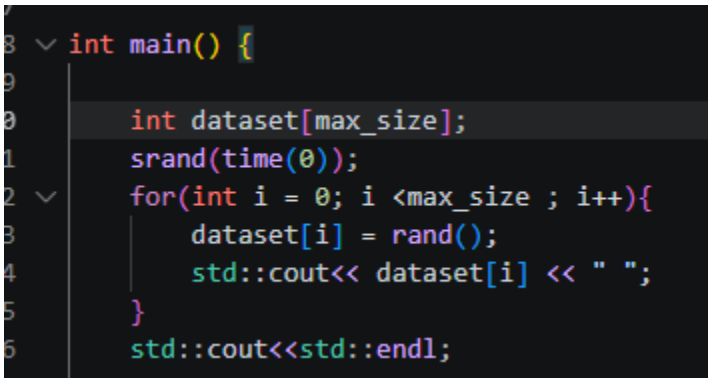
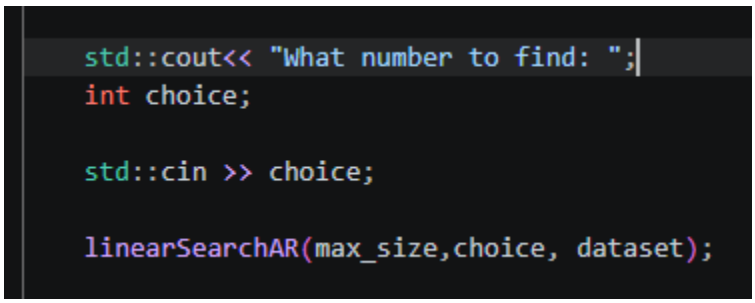
Screenshot	
Observations	The dataset is generated dynamically in main.cpp using rand() seeded with time(0). This populates an array of size 50 with random unsorted integers. Because the values are unsorted, this dataset is suitable for linear search but it must be sorted before binary search can be performed

Table 6-2a Linear Search for Arrays

Code	

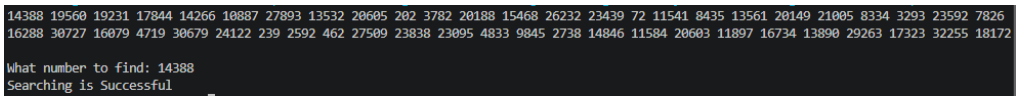
Output	
Observations	linearSearchAR prompts the user for a search value and scans through the randomly generated array element by element from index 0 to 49. Since the data is unsorted, linear search checks each element sequentially until a match is found or the end of the array is reached.

Table 6-2b Linear Search for Linked List

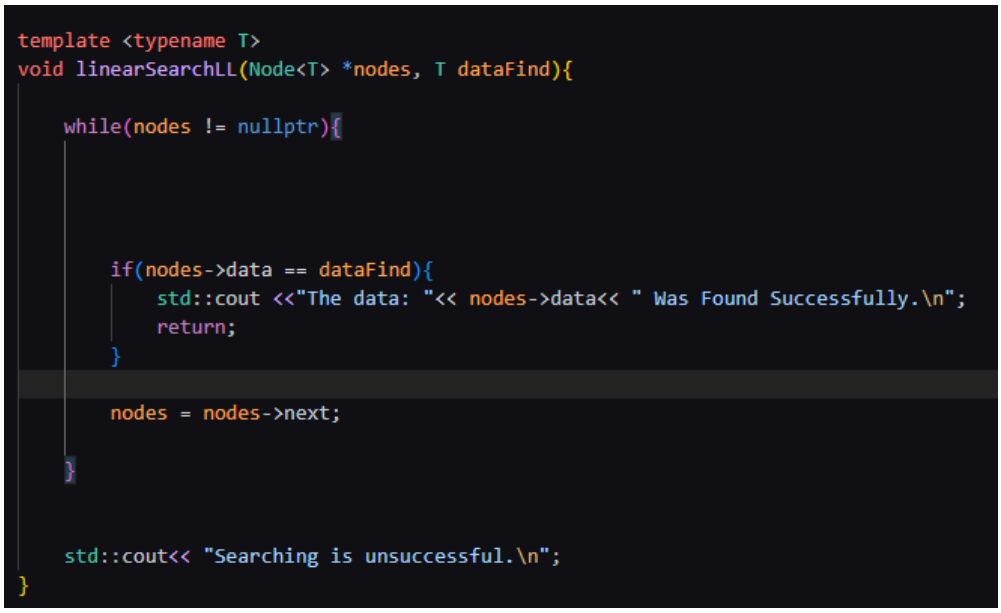
Code	
Output	
Observations	Linear search on a linked list traverses node pointers sequentially using nodes = nodes->next. Searching continues until the element is located or the pointer reaches nullptr. Time complexity is O(n) worst-case

Table 6-3a Binary Search for Arrays

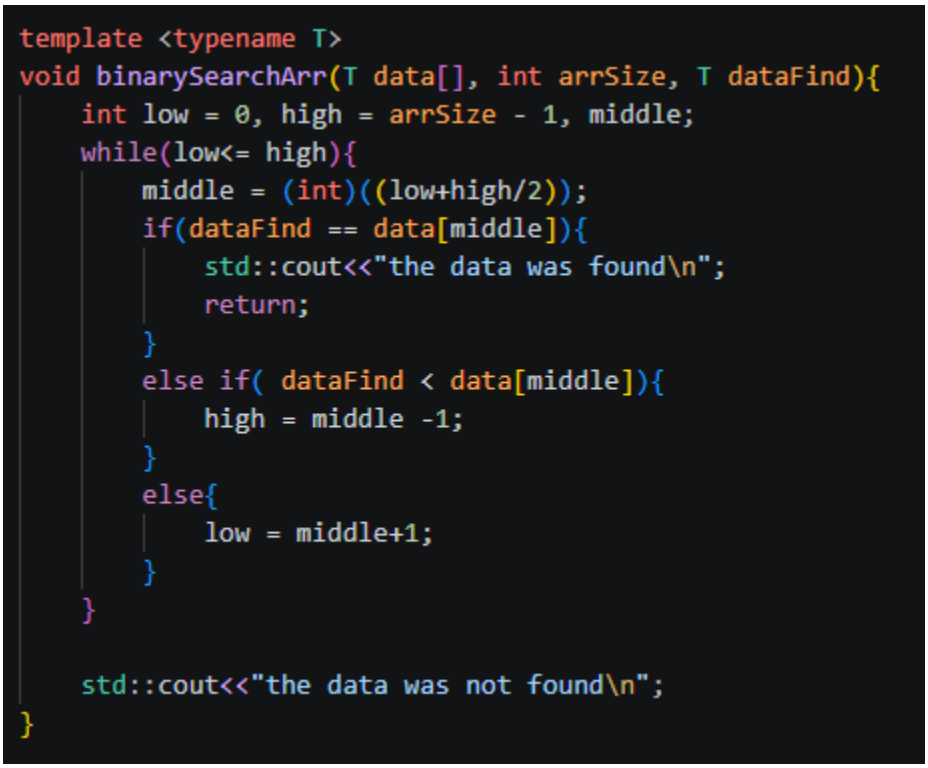
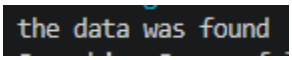
Code	
Output	
Observations	Binary search on an array utilizes direct indexing to divide the search range in half each iteration. Time Complexity is O(log n) making it significantly faster than linear search on large datasets

Table 6-3b Binary Search for Linked List

Code

```
73
74  template <typename T>
75
76  Node<T>* getMiddle(Node<T> *start, Node<T>* last){
77      Node<T>* slow = start;
78      Node<T>* fast = start;
79
80      while(fast != last && fast->next != last){
81
82          slow = slow ->next;
83
84          fast = fast ->next->next;
85
86      }
87
88      return slow;
89
90  }
91
```

```cpp
template <typename T>
void binarySearchLL(Node<T>* start, T dataFind){

    Node<T>* last = nullptr;

    while(start != last){

    Node<T> *middle = getMiddle(start, last);


    if(middle->data == dataFind){
        std::cout<< "Searching Successful\n";
        return;
    }

    else if(middle->data >dataFind){
        last = middle;
    }
    else{
        start = middle->next;
    }

    }

    std::cout<<"Searching is Unsuccessful\n";

}
#endif
```

Output	Searching Successful Searching is Unsuccessful Searching Successful
Observations	Binary search on a linked list requires finding the middle node using the two pointer technique because linked lists lack random index access. Traversing to find the middle node adds O(n) overhead per step, resulting in an overall time complexity of O(n log n).

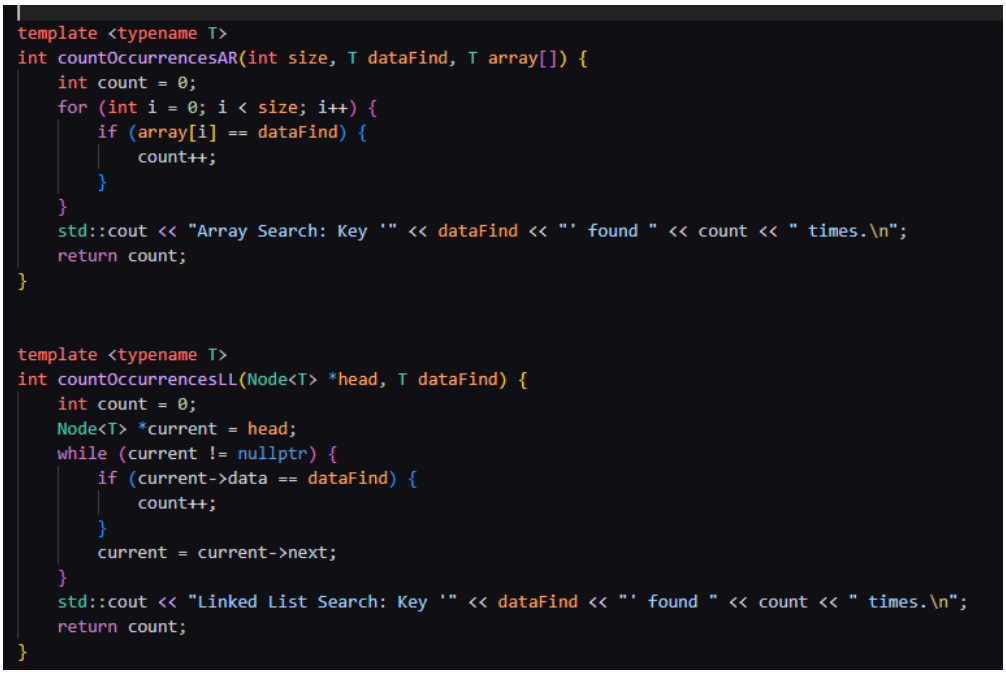
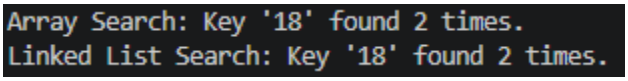
2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1:

Code	`int prob1Arr[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};`
	`int size1 = 10;`
	`int key1 = 18;`
	
	`linearSearchAR(size1, key1, prob1Arr);`
	`linearSearchLL(head, key1);`

Output	`Searching is Successful` `The data: 18 Was Found Successfully.`
Observations	Sequential search checks elements starting from index 0. Target value was 18 and is located at index 1 on the second comparison, stopping search execution early with a total comparison count of 2

Problem 2:

Code	
Output	
Observations	Unlike standard linear search which returns on first match, the modified algorithm traverses the entire structure to increment a counter every time array [i] == k or current->data == k

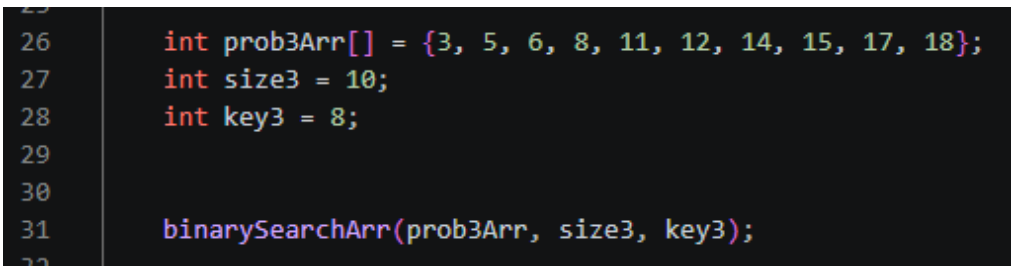
```
template <typename T>
int countOccurrencesAR(int size, T dataFind, T array[]) {
    int count = 0;
    for (int i = 0; i < size; i++) {
        if (array[i] == dataFind) {
            count++;
        }
    }
    std::cout << "Array Search: Key '" << dataFind << "' found " << count << " times.\n";
    return count;
}


template <typename T>
int countOccurrencesLL(Node<T> *head, T dataFind) {
    int count = 0;
    Node<T> *current = head;
    while (current != nullptr) {
        if (current->data == dataFind) {
            count++;
        }
        current = current->next;
    }
    std::cout << "Linked List Search: Key '" << dataFind << "' found " << count << " times.\n";
    return count;
}
```

```
21
22      countOccurrencesAR(size1, key1, prob1Arr);
23      countOccurrencesLL(head, key1);
```

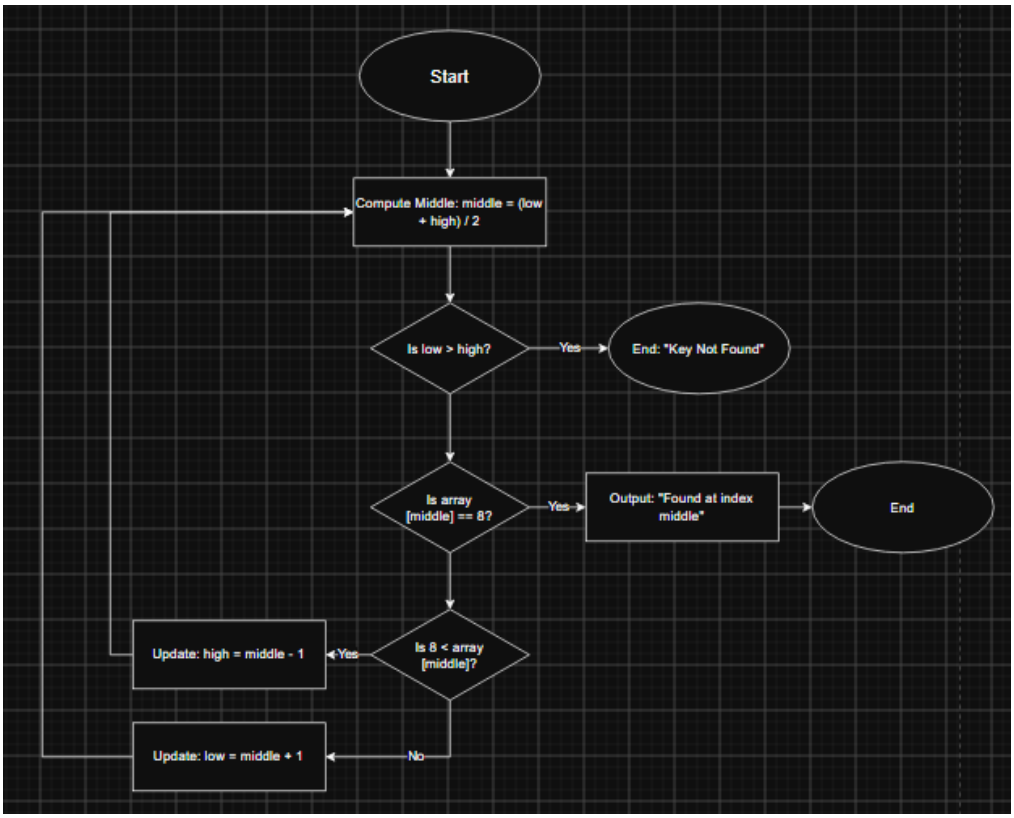
```
Array Search: Key '18' found 2 times.
Linked List Search: Key '18' found 2 times.
```

Problem 3:

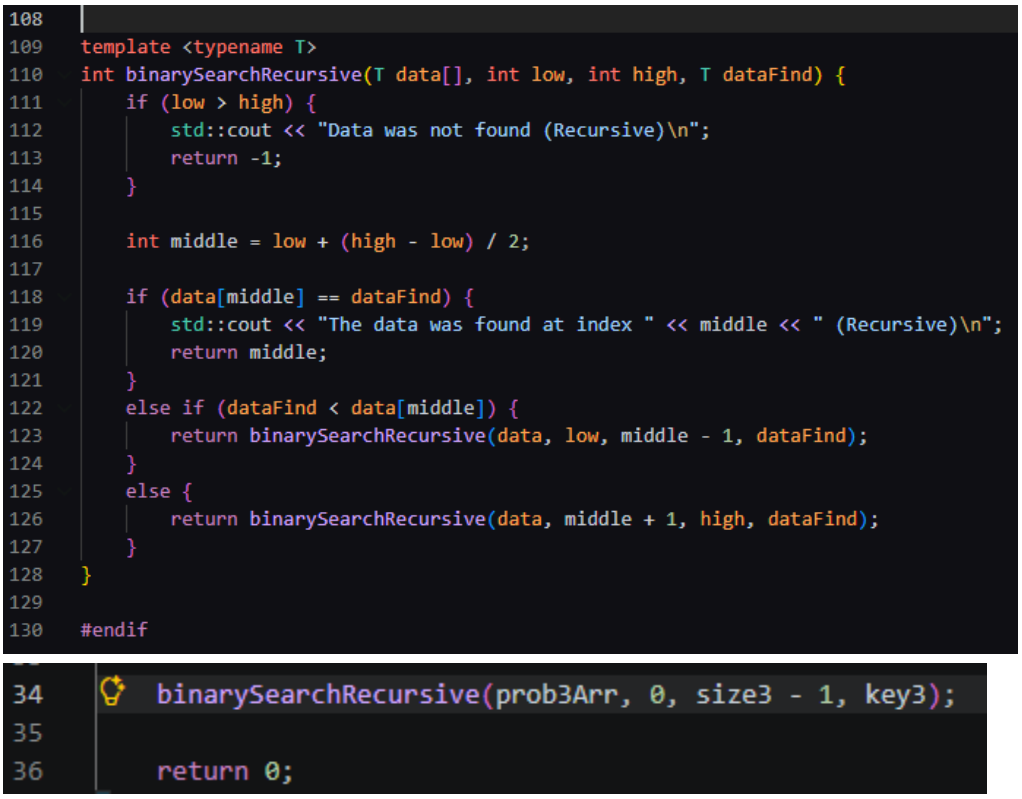
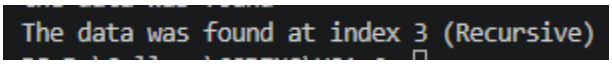
Code	
Output	

```
26      int prob3Arr[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
27      int size3 = 10;
28      int key3 = 8;
29
30
31      binarySearchArr(prob3Arr, size3, key3);
```

```
the data was found
```

Flowchart	
Observations	The array size is 10. The search computes middle and narrows the range in 4 steps: Pass 1 (mid=4, val=11 search left), Pass 2 (mid=1, val=5, search right) Pass 3 (mid=2, val= 6, search right), Pass 4 (mid=3, val = 8, match found).

Problem 4:

Code	
Output	
Observations	The recursive implementation replaces the while(low <= high) loop with base cases (low > high) for failure, data[middle] == dataFind for success) and recursive call steps that adjust boundaries (middle - 1 or middle + 1).

Conclusion

I learned linear search, binary search in arrays and linked lists in C++ and understand that linear search in arrays takes O(n) time when done on an unsorted array while binary search is O(n) for an array with contiguous elements, but the additional time taken to traverse the array to find middle elements while performing binary search in linkedlist.

The implementation procedure illustrated the simplicity of sequential array indexing and updating the node pointers in linear search, while binary search requires the data to be sorted and the use of fast and slow pointer methods to split the search interval.

The supplementary exercise of changing the linear search functions to count duplicate entries involved doing the full scan, rather than breaking out of the loop early, and the recursive rewrite of binary search showed how base cases and recursive calls can elegantly replace iterative while loops.

Overall, I successfully performed the activity in debugging, testing, and modifying all searching algorithms, still

can keep doing better optimizing the pointer traversal of lists and writing better recursive versions.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, concept clarification, formatting, algorithm analysis, and grammar refinement.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used alongside the instructor's lecture video as a supportive learning and drafting resource. The lecture video was referenced to ensure alignment with the instructor's expected coding style and templates. AI assisted in clarifying recursive logic verifying step by step binary search traces and structuring written report sections. All final codes was written, integrated, compiled, tested and verified by me. The AI did not replace my original work or authorial control over the completed activity**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.